



Tecnológico de Monterrey

Programacion de estructura de datos y algoritmos fundamentales

Profesor: David Alonso Cantu Delgado

Julio Eduardo Rodríguez Ruiz | A00839165

1. Importancia de ordenar y buscar en logs

En el contexto de la ciberseguridad y la administración de servidores, los archivos de bitácora (logs) son la principal fuente de verdad para rastrear la actividad de los usuarios y diagnosticar incidentes. En los archivos provistos, tenemos miles de registros con alertas críticas como intentos de "SQL injection" o "Malware Detection Alerts". Sin embargo, en un entorno real distribuido, los logs rara vez llegan perfectamente sincronizados debido a la latencia de la red o desfases de reloj entre servidores.

Ordenar estos registros cronológicamente es fundamental porque permite reconstruir la línea de tiempo exacta de un ataque. Por otro lado, la búsqueda eficiente es igual de crítica. Cuando ocurre un incidente de seguridad, los administradores no pueden revisar millones de líneas secuencialmente; necesitan extraer el segmento de tiempo exacto donde ocurrió la brecha de forma casi instantánea, lo cual solo es matemáticamente posible si los datos están previamente ordenados y se aplican algoritmos de búsqueda logarítmica.

2. Comparación conceptual de algoritmos

Para este proyecto se implementaron distintos enfoques que demuestran comportamientos muy diferentes según el estado inicial de los datos:

- **Selection Sort:** Su complejidad teórica siempre es $O(n^2)$ porque irremediablemente debe recorrer toda la porción no ordenada para encontrar el mínimo, sin importar si los datos ya están en orden. Su única fortaleza es que realiza el mínimo número de intercambios (swaps) posibles en memoria, pero es ineficiente para arreglos grandes.
- **Merge Sort:** Utiliza el enfoque en el que fracciona el arreglo hasta llegar a la unidad y luego fusionándolos. Su gran fortaleza es que garantiza una complejidad asintótica de $O(n \log n)$ en el mejor, peor y caso promedio. Es sumamente estable y predecible, aunque su debilidad es que requiere memoria extra $O(n)$ para los arreglos temporales durante la fusión.
- **Insertion Sort (El caso atípico):** Su complejidad teórica en el peor de los casos (datos invertidos o aleatorios) es $O(n^2)$. Sin embargo, su comportamiento cambia drásticamente con datos casi ordenados, como es el caso del archivo *log607-2.txt*. Debido a que su ciclo interno while se rompe en el momento en que encuentra que el elemento actual ya es mayor que el anterior, en un arreglo que solo tiene desfases locales mínimos, el ciclo interno apenas se ejecuta. Esto reduce su complejidad asintótica en la práctica a casi $O(n)$ en el mejor de los casos.

3. Interpretación de las mediciones empíricas

Antes de ejecutar los algoritmos, la predicción basada en la teoría indicaba que Selection Sort sería el más lento en todos los escenarios, Merge Sort mantendría un tiempo bajo y constante, e Insertion Sort tendría un desempeño pobre en el archivo desordenado pero superaría sus propias métricas en el archivo casi ordenado.

Al analizar los tiempos reales arrojados por la librería `<chrono>`, la teoría se confirma matemáticamente. En el archivo `log607-1.txt` (desordenado aleatoriamente), el *Insertion Sort* compite negativamente junto al *Selection Sort*, evidenciando su crecimiento cuadrático $O(n^2)$. En contraste, al procesar `log607-2.txt` (casi ordenado), el tiempo de ejecución del *Insertion Sort* cae en picada, comportándose de manera lineal $O(n)$ y ejecutándose en una fracción diminuta del tiempo que le toma al *Selection Sort*. El *Merge Sort*, fiel a su naturaleza logarítmica, mantiene tiempos de ejecución muy similares en ambos archivos, demostrando que no es sensible al nivel de ordenamiento inicial de los datos.

4. Uso crítico de la Inteligencia Artificial

Durante el desarrollo, utilicé la IA como un revisor de sintaxis y consultor lógico, aplicando la siguiente iteración de prompts:

Prompt 1: *"Genera una estructura en C++ para leer líneas de un archivo de log con fecha, hora, IP y mensaje. Además, necesito que convierta el mes de texto (Jan, Feb) a número para poder ordenarlo cronológicamente."*

- **Modificación realizada:** La IA sugirió guardar todo en variables enteras separadas y sobrecargar el operador comparando año, luego mes, luego día, etc. Modifiqué esta lógica para generar una única `long long claveOrdenamiento` (multiplicando los valores para unirlos en un solo número entero). Esto hizo que las comparaciones dentro de los bucles de ordenamiento fueran una simple evaluación matemática $O(1)$ directa, optimizando el rendimiento.

Prompt 2 (Identificación de error algorítmico): *"Escribe una función de búsqueda binaria en C++ para encontrar un rango de fechas en mi vector de registros de log."*

- **Detección y corrección del error:** La IA sugirió una implementación de búsqueda binaria estándar que, al encontrar la clave buscada, retornaba inmediatamente el índice de esa primera coincidencia `if (arr[medio].clave == buscado) return medio;`. Me di cuenta de que esto era algorítmicamente incorrecto para los requisitos del proyecto. El archivo `log607-2.txt` contiene múltiples timestamps duplicados a propósito. Si utilizaba el código de la IA, la búsqueda binaria caería aleatoriamente en medio de un bloque de registros idénticos, omitiendo los duplicados que estuvieran a la izquierda o a la derecha, lo cual corrompería la exportación final.
- **Solución:** Rechacé la función de la IA y desarrollé manualmente dos variantes (`busquedaBinariaInicio` y `busquedaBinariaFin`). Modifiqué la condición matemática para que, incluso tras encontrar una coincidencia, la función inferior continuara buscando hacia la izquierda (`fin = medio - 1`) y la superior hacia la derecha (`inicio = medio + 1`). De esta manera, resolví el

borde obligatorio de los datos duplicados manteniendo intacta la eficiencia $O(\log n)$.